

Professores

Celso Giusti

Daniel Filho

Felipe Gregate

Marlon Rodrigues

Clean Code



Objetivos

Compreender e aplicar os princípios de Clean Code para escrever códigos mais claros, legíveis e fáceis de manter.

Por que Clean Code importa?

O código é lido muito mais vezes do que é escrito. Isso significa que a forma como escrevemos afeta diretamente a produtividade, o tempo de manutenção e até o clima da equipe.

Quando o código é confuso, cada alteração vira um risco, e tarefas simples tornam-se demoradas.

Em resumo, código limpo:

- reduz erros,
- facilita evolução,
- melhora a colaboração,
- e torna o trabalho mais leve.

O Propósito do Clean Code

Clean Code não é luxo — é sobrevivência. Ele torna o software mais fácil de entender, modificar e evoluir.

Escrever código limpo é escrever para pessoas, não apenas para máquinas.

Objetivos principais:

- reduzir retrabalho;
- permitir refatorações seguras;
- facilitar onboarding de novos devs.

Clean Code é fundamentado em 6 pilares.

Pilar 1: Nomes Significativos

Nomes revelam intenção. Quando você dá nomes claros, o código praticamente se explica sozinho.

Variáveis, funções e classes devem comunicar seu papel sem esforço.

Boas práticas:

- evite nomes curtos e genéricos;
- prefira clareza a economia;
- funções devem ser verbos;
- classes devem ser substantivos.

Pilar 1: Exemplos de bons nomes

Ruins: *d, x, lista, handle()*

Bons: *diasParaExpiracao,*
produtoEncontrado,
calcularImpostoSobreVenda(),
GerenciadorDeEstoque.

Um bom nome economiza horas de leitura futura.

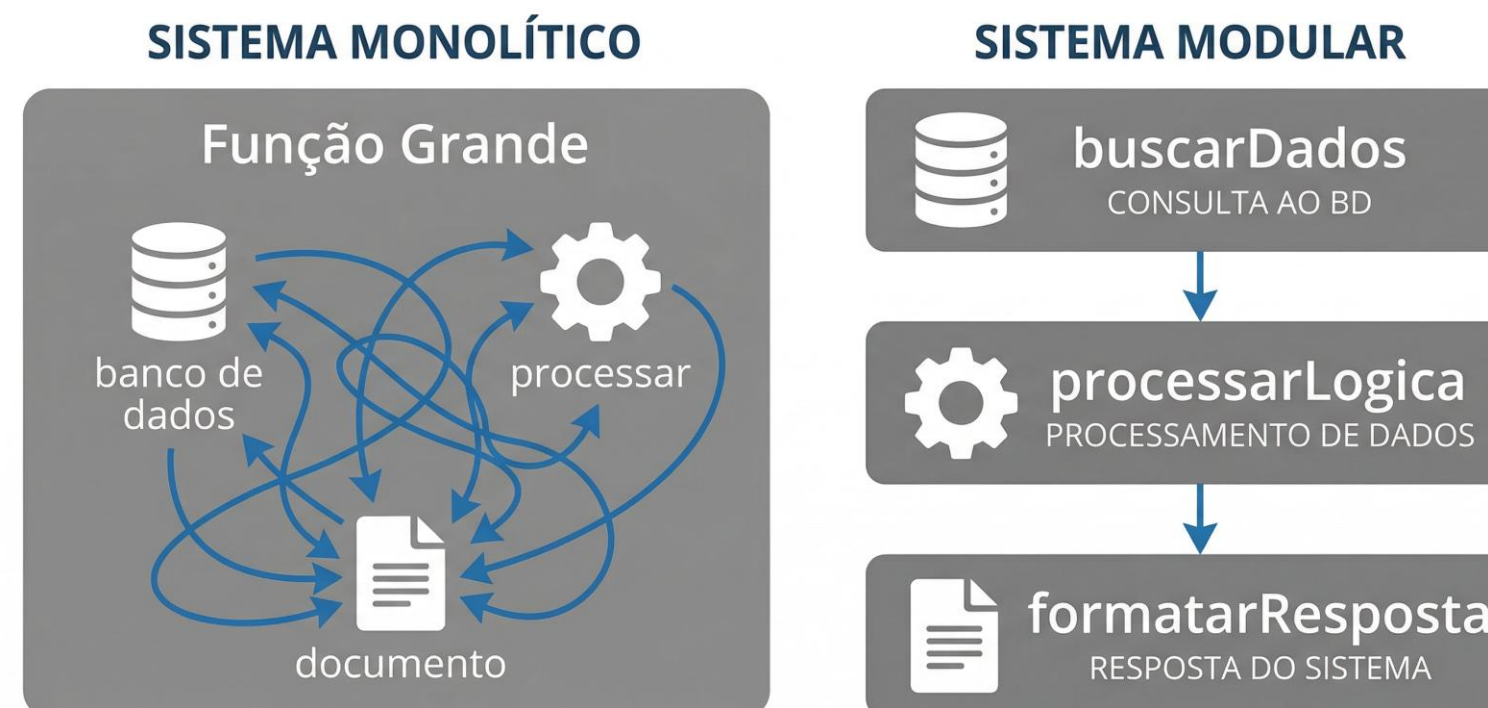
Pilar 2: Funções Pequenas e Focadas

Funções devem fazer uma única coisa e fazê-la bem. Quando uma função acumula responsabilidades, ela se torna difícil de entender e impossível de testar isoladamente.

Evite funções que misturam:

- acesso a banco,
- regra de negócio,
- formatação,
- tratamento de erros,
- validações.

MONOLÍTICO VS. MODULAR: ESTRUTURA DE FUNÇÃO



Dividir funções cria clareza e permite evolução sem dor.

Pilar 2: Níveis de Abstração

Funções devem trabalhar em apenas um nível de detalhe.

Funções mais “altas” coordenam, enquanto funções “baixas” executam tarefas específicas.

Exemplo de fluxo saudável:

- buscarDados()
- processarLogica()
- formatarResposta()

Cada uma cumpre um papel claro.

Pilar 3: Comentários Bem Usados

Comentários bons explicam decisões. Comentários ruins descrevem o óbvio. O objetivo não é comentar mais, mas **comentar melhor**.

Comentários ruins:

- explicam algo evidente;
- descrevem código ruim;
- são desorganizados.

Comentários úteis:

- justificam decisões não óbvias;
- explicam contexto de negócio;
- registram TODOs.

Pilar 3: Código Expressivo

Antes de comentar um trecho difícil, pergunte a si mesmo:

“Eu consigo reescrever isso para ficar autoexplicativo?”

Na maioria das vezes, extrair para uma função com bom nome é suficiente.

Exemplo simples:

Em vez de um comentário, crie:

calcularTotalComImpostos().

Pilar 4: Formatação e Padrões

Um código bem formatado é mais fácil de ler, revisar e manter.

Padrões evitam que cada desenvolvedor escreva “de um jeito”.

Ferramentas recomendadas:

ESLint → qualidade e estilo

Prettier → formatação automática

A consistência melhora todo o projeto.

Pilar 4: Por que padronizar?

Quando estilos diferentes convivem no mesmo arquivo, o código fica cansativo.

Com padronização, você reduz a necessidade de “entender o estilo do autor” e foca no que importa: o conteúdo.

Benefícios:

- leitura rápida,
- menos ruído visual,
- fácil manutenção.

Pilar 5: Tratamento de Erros

Erros não devem ser ignorados. Um *catch* vazio é um dos maiores perigos do back-end.

Tratamento limpo envolve:

- capturar erros reais,
- registrar informações úteis,
- responder adequadamente ao cliente,
- nunca continuar o fluxo se os dados ficaram inconsistentes.

Um sistema robusto trata erros com clareza.

Pilar 6: Organização Estrutural

A organização do projeto ajuda qualquer desenvolvedor a navegar pelo código.

Cada parte do back-end tem um papel específico.

Estrutura recomendada:

- routes/ — recebe requisições
- controllers/ — coordena a operação
- services/ — lógica de negócio
- models/ — comunicação com banco

Essa divisão mantém o projeto claro e modular.

Pilar 6: Fluxo ideal no back-end

Quando uma requisição chega, o fluxo é natural e previsível:

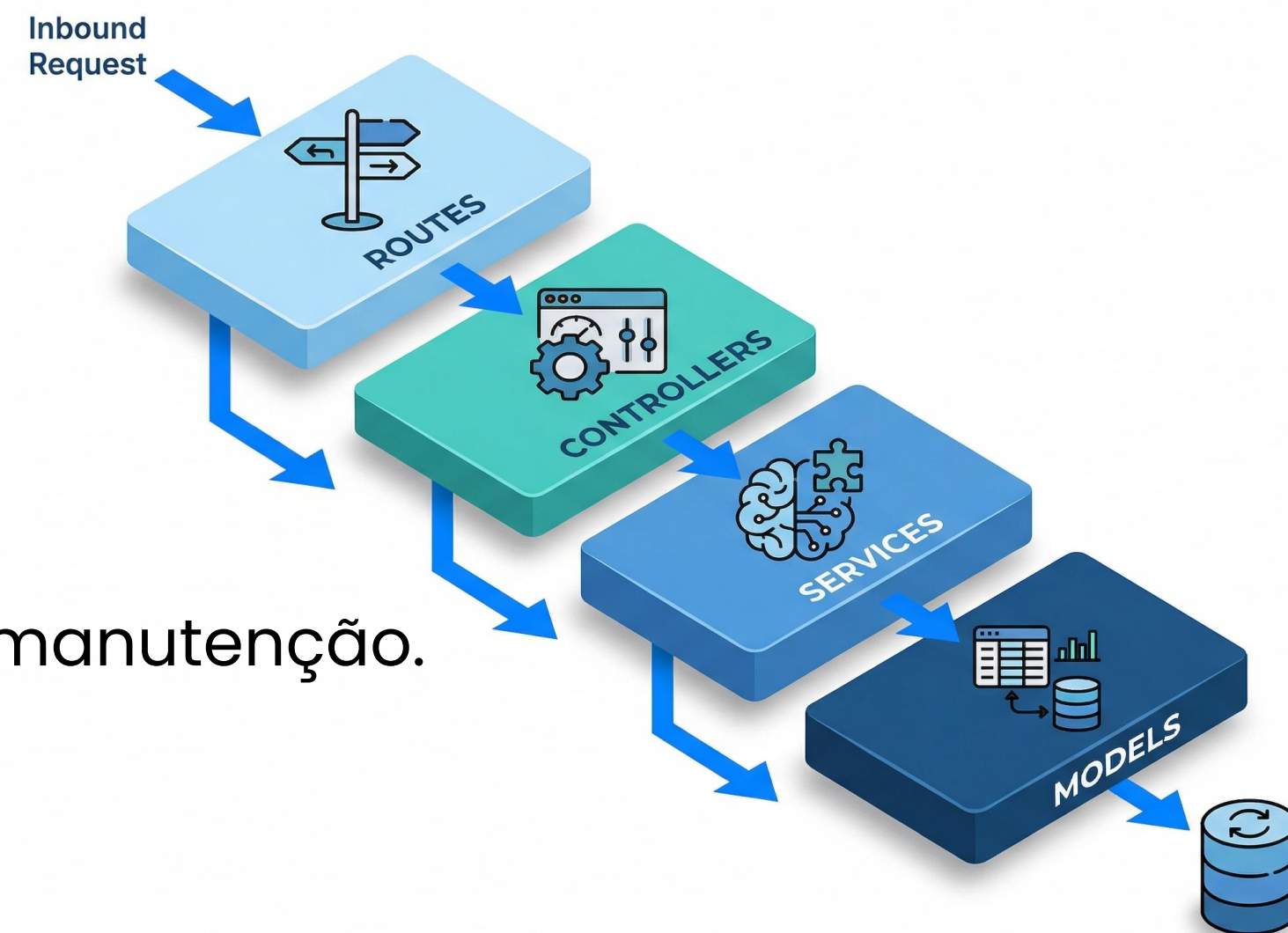
Route identifica o endpoint

Controller interpreta e delega

Service executa regras de negócio

Model acessa o banco

Esse fluxo organizado facilita testes e manutenção.



Resumo da aula

- ✓ Clean Code – Conjunto de 6 pilares para design de código limpo e escalável.
- ✓ 1. Nomes significativos
- ✓ 2. *Funções pequenas e focadas*
- ✓ 3. *Comentários bem utilizados*
- ✓ 4. *Formatação e padrões*
- ✓ 5. *Tratamento de erros*
- ✓ 6. *Organização estrutural*

GIUSTI, Celso Rodrigo. **Técnicas de Codificação e Design para Sistemas Back-End**. São Paulo: SENAI-SP, [s.d.].

MARTIN, Robert C. **Código Limpo: Habilidades Práticas do Software Ágil**. Rio de Janeiro: Alta Books, 2009.

SENAI-SP. **Plano de Curso: Técnico em Desenvolvimento de Sistemas**. São Paulo: Departamento Regional de São Paulo, 2023.

Parte do conteúdo dessa apresentação foi gerada por IA com o propósito de deixar a apresentação mais dinâmica.



Escola SENAI “Italo Bologna”

Av. Goiás, 139 – Itu/SP

Telefone

(11) 2396-1999

Instagram

@senai.itu

Facebook

/senai.itu

Site

<https://sp.senai.br/unidade/itu/>